

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY  
UNIVERSITY OF TECHNOLOGY  
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



**Microprocessor and Microcontroller(CO3009)**

---

**ASSIGNMENT**

**TRAFFIC LIGHT**

---

GVDH: Mr. Lê Trọng Nhân

Mr. Cao Tiến Đạt

Sinh viên: Lê Quang Minh MSSV 2212047

HO CHI MINH CITY, December 2024



## Contents

|          |                                                 |          |
|----------|-------------------------------------------------|----------|
| <b>1</b> | <b>GITHUB code</b>                              | <b>2</b> |
| <b>2</b> | <b>Giới thiệu đề tài</b>                        | <b>2</b> |
| <b>3</b> | <b>Tạo một dự án STM32</b>                      | <b>2</b> |
| <b>4</b> | <b>Máy trạng thái</b>                           | <b>3</b> |
| <b>5</b> | <b>Hiện thực</b>                                | <b>6</b> |
| 5.1      | Các mã nguồn trong dự án được sử dụng . . . . . | 6        |
| 5.2      | Giới thiệu Scheduler . . . . .                  | 6        |
| 5.3      | Giới thiệu LCD . . . . .                        | 9        |

## 1 GITHUB code

Link code Assignment [Code Assignment](#)

## 2 Giới thiệu đề tài

Thiết kế một hệ thống đèn giao thông sử dụng vi điều khiển STM32F103RB gồm hai chế độ là Automatic và Manual, sử dụng 2 nút nhấn để chuyển chế độ và 1 màn hình LCD16x2 để hiển thị.

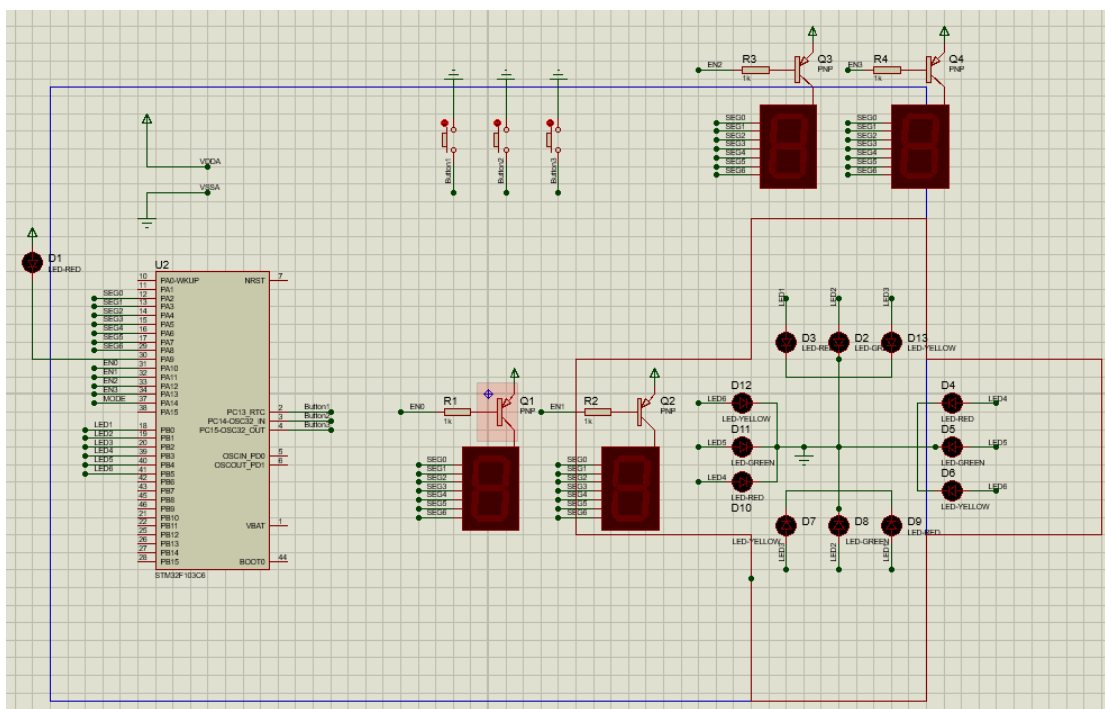


Figure 1: Mô phỏng trên Proteus

## 3 Tạo một dự án STM32

Định nghĩa các chân trên stm32 được sử dụng.

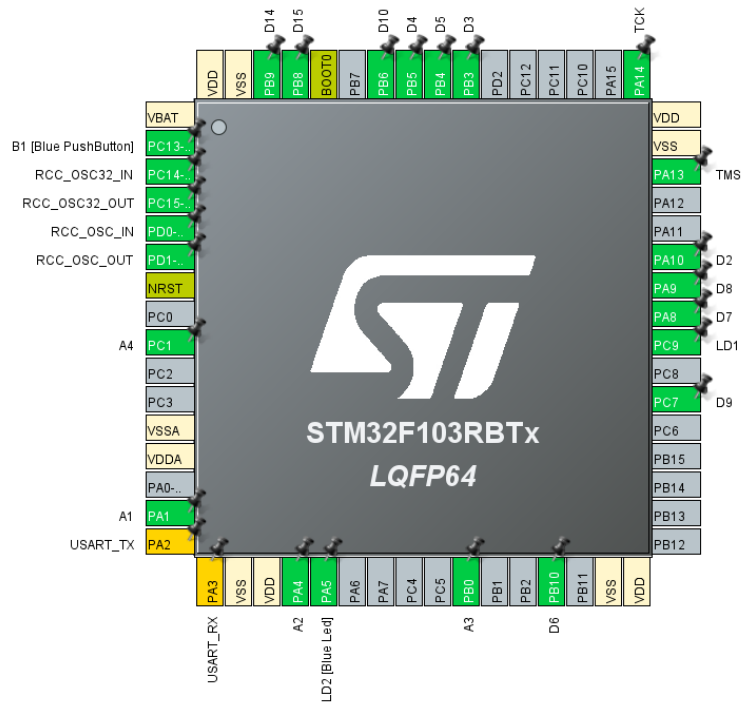


Figure 2: File F103RB.ioc của Assignment

## 4 Máy trạng thái

Máy trạng thái cho toàn bộ hệ thống Traffic light.

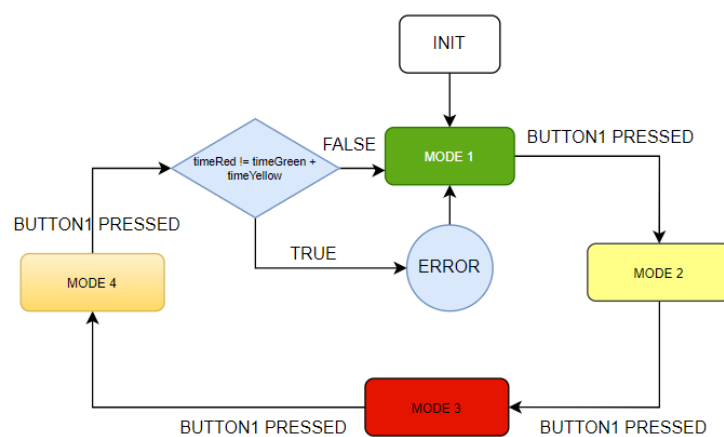


Figure 3: Máy trạng thái

Máy trạng thái cho Traffic light mode 1.

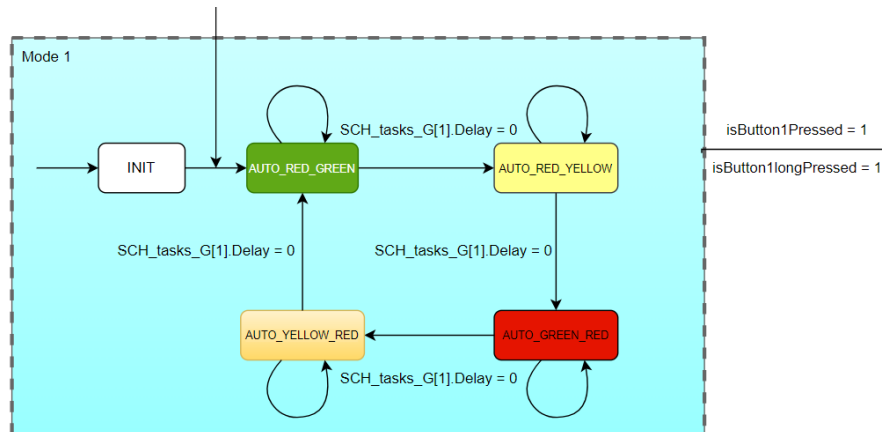


Figure 4: Máy trạng thái mode 1

Máy trạng thái cho Traffic light mode 2.

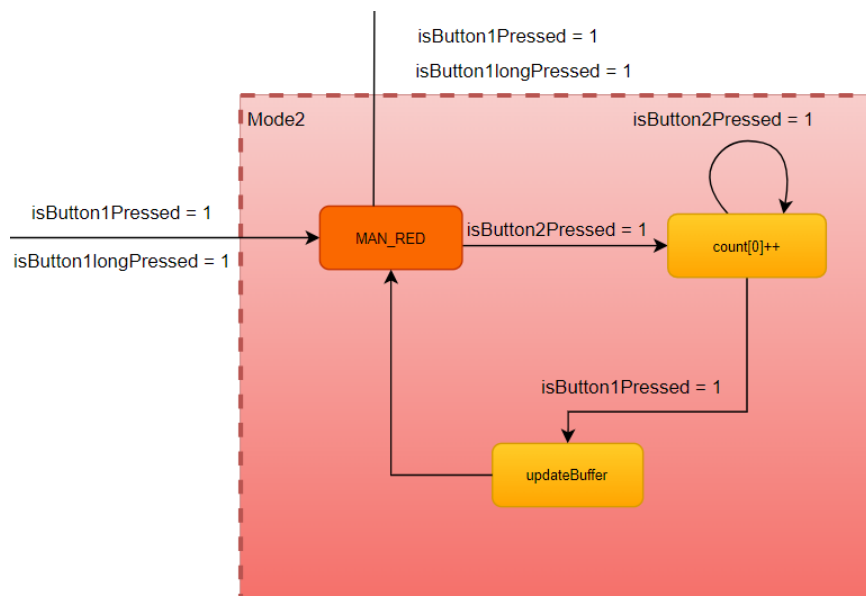


Figure 5: Máy trạng thái mode 2

Máy trạng thái cho Traffic light mode 3.

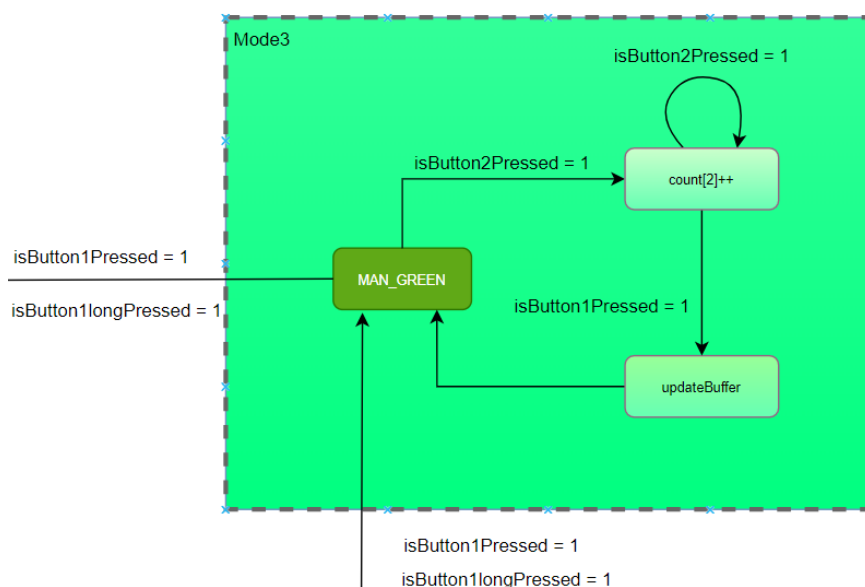


Figure 6: Máy trạng thái mode 3

Máy trạng thái cho Traffic light mode 4.

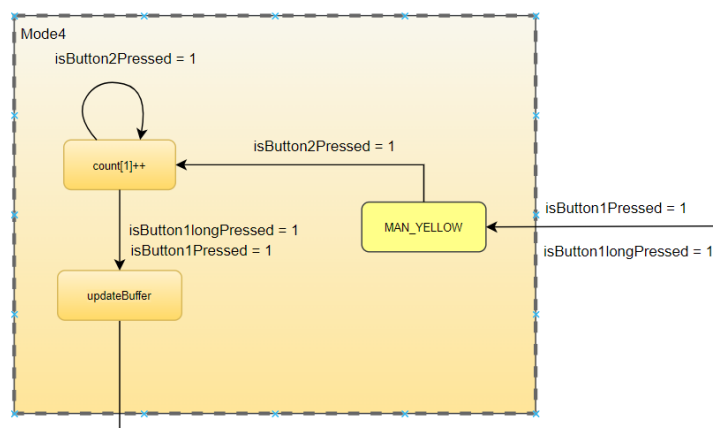


Figure 7: Máy trạng thái mode 4

## 5 Hiện thực

### 5.1 Các mã nguồn trong dự án được sử dụng

- main.c: thực hiện gọi hàm để xử lý dự án.
- scheduler.c: hiện thực cooperative schedulling.
- fsm\_manual.c: xử lý cho trạng thái MANUAL.
- fsm\_automatic.c: xử lý cho trạng thái AUTOMATIC.
- global.c: khai báo thư viện, các biến và hàm được sử dụng cho toàn bộ.
- led\_traffic.c: điều khiển hoạt động đèn led.
- liquidcrystal\_i2c.c: giao tiếp với LCD.
- button.c: xử lý nút nhấn (chống run, nhấn đúp, ...).
- scan7led.c: xử lý quét led 7 đoạn.

### 5.2 Giới thiệu Scheduler

Quản lý thực hiện hệ thống, quản lý thực hiện các hàm được sử dụng

Listing 1: Code in scheduler.c

```
1 #include "scheduler.h"
2
3 sTasks SCH_tasks_G[SCH_MAX_TASKS];
4 uint8_t is_add[SCH_MAX_TASKS] = {0};
5 uint32_t numTask = 0;
6
7 void SCH_Init(void){ // checked
8     for(uint8_t i = 0; i < SCH_MAX_TASKS; i++){
9         SCH_Delete(i);
10        is_add[i] = 0;
```

```
11     }
12     numTask = 0;
13 }
14 // modifying
15 void SCH_Add_Task(void (*pFunction)(), uint32_t DELAY,
16                 uint32_t PERIOD, uint32_t TaskID){
17     if(numTask >= SCH_MAX_TASKS) return;
18
19     uint32_t total = 0;
20     uint32_t i = 0;
21     uint32_t newDelay = DELAY;
22
23     // Kim tra v tr th m t c v
24     for(; i < numTask; i++){
25         if(SCH_tasks_G[i].pTask){
26             total += SCH_tasks_G[i].Delay;
27             if(total > newDelay){
28                 newDelay -= (total - SCH_tasks_G[i].Delay);
29                 break;
30             }
31         }
32     }
33
34     if (i == numTask) {
35         newDelay -= total;
36     }
37
38     for(int j = numTask; j > i; j--){
39         SCH_tasks_G[j] = SCH_tasks_G[j - 1];
40     }
41     SCH_tasks_G[i + 1].Delay -= newDelay;
42     SCH_tasks_G[i].pTask = pFunction;
43     SCH_tasks_G[i].Delay = newDelay;
44     SCH_tasks_G[i].Period = PERIOD;
45     SCH_tasks_G[i].RunMe = 0;
46     SCH_tasks_G[i].TaskID = TaskID;
```



```
47
48     numTask++;
49     is_add[numTask - 1] = 1;
50 }
51 void SCH_Update(void){ //checked
52     if(is_add[0]){
53         if(SCH_tasks_G[0].Delay > TICK){
54             SCH_tasks_G[0].Delay -= TICK;
55         }
56         else {
57             SCH_tasks_G[0].Delay = 0;
58             SCH_tasks_G[0].RunMe = 1;
59         }
60     }
61 }
62 void SCH_Dispatch_Tasks(void){ //checked
63     if(numTask == 0) return;
64     while(SCH_tasks_G[0].RunMe == 1 || SCH_tasks_G[0].Delay == 0){
65
66         (*SCH_tasks_G[0].pTask)();
67
68         sTasks tempTask = SCH_tasks_G[0];
69         SCH_Delete_Task(SCH_tasks_G[0].TaskID);
70         if(tempTask.Period != 0){
71             SCH_Add_Task(tempTask.pTask, tempTask.Period,
72                 tempTask.Period, tempTask.TaskID);
73         }
74     }
75 }
76 void SCH_Delete(uint32_t TASK_INDEX){
77     SCH_tasks_G[TASK_INDEX].pTask = 0 ;
78     SCH_tasks_G[TASK_INDEX].Delay = 0;
79     SCH_tasks_G[TASK_INDEX].Period = 0;
80     SCH_tasks_G[TASK_INDEX].RunMe = 0;
81     SCH_tasks_G[TASK_INDEX].TaskID = 0;
82 }
```

```
83 uint8_t SCH_Delete_Task(uint32_t id){ //checked
84     if(numTask == 0){
85         return 0;
86     }
87     uint8_t success = 0;
88     uint32_t i;
89     for(i = 0; i < numTask; i++){
90         if(id == SCH_tasks_G[i].TaskID){
91
92             break;
93         }
94     }
95     uint32_t delay = SCH_tasks_G[i].Delay;
96     for (uint32_t j = i; j < numTask - 1; j++){
97         SCH_tasks_G[j] = SCH_tasks_G[j + 1];
98         SCH_tasks_G[j].Delay = SCH_tasks_G[j].Delay + delay;
99     }
100     success = 1;
101     is_add[numTask] = 0;
102     numTask--;
103     return success;
104 }
```

### 5.3 Giới thiệu LCD

Thực hiện khởi tạo và điều khiển LCD.

Listing 2: Code in liquidcrystal\_i2c.c

```
1 #include "liquidcrystal_i2c.h"
2 extern I2C_HandleTypeDef hi2c1; // change your handler here accordingly
3
4 #define SLAVE_ADDRESS_LCD (0x21 << 1) // change this according to ur setup
5
6 void lcd_send_cmd (char cmd)
7 {
```

```
8  char data_u, data_l;
9  uint8_t data_t[4];
10 data_u = (cmd&0xf0);
11 data_l = ((cmd<<4)&0xf0);
12 data_t[0] = data_u|0x0C;  //en=1, rs=0
13 data_t[1] = data_u|0x08;  //en=0, rs=0
14 data_t[2] = data_l|0x0C;  //en=1, rs=0
15 data_t[3] = data_l|0x08;  //en=0, rs=0
16 HAL_I2C_Master_Transmit (&hi2c1, SLAVE_ADDRESS_LCD,(uint8_t *) data_t, 4, 100);
17 }
18
19 void lcd_send_data (char data)
20 {
21     char data_u, data_l;
22     uint8_t data_t[4];
23     data_u = (data&0xf0);
24     data_l = ((data<<4)&0xf0);
25     data_t[0] = data_u|0x0D;  //en=1, rs=0
26     data_t[1] = data_u|0x09;  //en=0, rs=0
27     data_t[2] = data_l|0x0D;  //en=1, rs=0
28     data_t[3] = data_l|0x09;  //en=0, rs=0
29     HAL_I2C_Master_Transmit (&hi2c1, SLAVE_ADDRESS_LCD,(uint8_t *) data_t, 4, 100);
30 }
31
32 void lcd_init (void) {
33     lcd_send_cmd (0x33); /* set 4-bits interface */
34     lcd_send_cmd (0x32);
35     HAL_Delay(50);
36     lcd_send_cmd (0x28); /* start to set LCD function */
37     HAL_Delay(50);
38     lcd_send_cmd (0x01); /* clear display */
39     HAL_Delay(50);
40     lcd_send_cmd (0x06); /* set entry mode */
41     HAL_Delay(50);
42     lcd_send_cmd (0x0c); /* set display to on */
43     HAL_Delay(50);
```



```
44  lcd_send_cmd (0x02); /* move cursor to home and set data address to 0 */
45  HAL_Delay(50);
46  lcd_send_cmd (0x80);
47  }
48
49  void lcd_send_string (char *str)
50  {
51      while (*str) lcd_send_data (*str++);
52  }
53
54  void lcd_clear_display (void)
55  {
56      lcd_send_cmd (0x01); //clear display
57  }
58
59  void lcd_goto_XY (int row, int col)
60  {
61      uint8_t pos_Addr;
62      if(row == 1)
63      {
64          pos_Addr = 0x80 + row - 1 + col;
65      }
66      else
67      {
68          pos_Addr = 0x80 | (0x40 + col);
69      }
70      lcd_send_cmd(pos_Addr);
71  }
```

**END ASSIGNMENT**